

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **2**
Topic : **6**

NoSQL Systems to handle Big data

Objectives

This session will give the knowledge about

- NoSQL Systems to handle Big data

NoSQL Systems to handle Big data

Understanding your big data is important in determining the best solution.

Look at **four of the most popular ways NoSQL systems handle big data challenges**. Understanding these techniques is important when you're evaluating any NoSQL system.

- Moving queries to the data, not data to the queries
- Using hash rings to evenly distribute data on a cluster
- Using replication to scale reads
- Letting the database distribute queries evenly to data nodes

Moving queries to the data, not data to the queries

Most NoSQL systems use commodity processors that **each hold a subset of the data on their local shared-nothing drives** except large graph databases.

When a client wants to send a general query to all nodes that hold data, **it's more efficient to send the query to each node than it is to transfer large datasets to a central processor.**

This may seem obvious, but it's amazing how many traditional databases still can't distribute queries and aggregate query results.

This simple rule helps you understand how NoSQL databases can have dramatic performance advantages over systems that weren't designed to distribute queries to the data nodes.

Moving queries to the Data

Consider an RDBMS that has tables distributed over two different nodes.

In order for the SQL query to work, information about rows on one table must all be moved across the network to the other node. Larger tables result in more data movement, which results in slower queries. Think of all the steps involved.

The tables can be extracted, serialized, sent through the network interface, transmitted over networks, reassembled, and then compared on the server with the SQL query.

Keeping all the data within each data node in the form of logical documents means that only the query itself and the final result need to be moved over a network. This keeps your big data queries fast.

Using hash rings to evenly distribute data on a cluster

One of the most challenging problems with distributed databases is **figuring out a consistent way of assigning a document to a processing node.**

Using a **hash ring technique** to evenly distribute big data loads over many servers with a randomly generated 40-character key is a good way to evenly distribute a network load.

Hash rings are common in big data solutions because **they consistently determine how to assign a piece of data to a specific processor**

Hash Rings

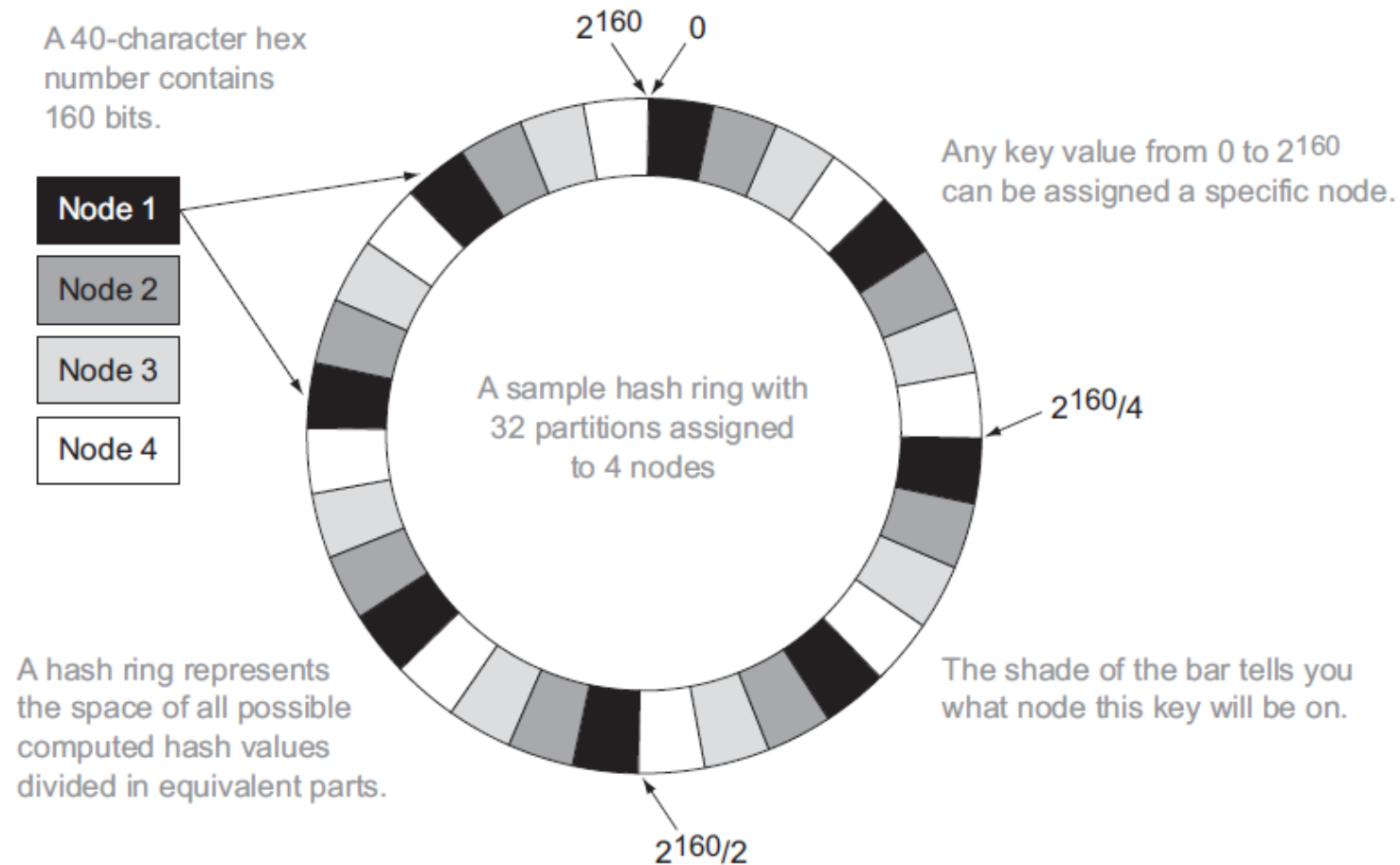
Hash rings take the **leading bits of a document's hash value** and use this to **determine which node the document should be assigned**.

This allows any node in a cluster to know what node the data lives on and how to adapt to new assignment methods as your data grows.

Partitioning keys into ranges and assigning different key ranges to specific nodes is known as **keyspace management**.

Most NoSQL systems, including MapReduce, use keyspace concepts to manage distributed computing problems.

Sample Hash Ring with four nodes



Hash Rings

A hash ring uses the concept of hashing, consistent hashing, and key-value stores to assign an item of data to a specific node in a NoSQL database cluster.

From figure, each input will be assigned to a node based on the 40-character random key.

One or more nodes in your cluster will be responsible for storing this key-to-node mapping algorithm.

As your database grows, you'll update the algorithm so that each new node will also be assigned some range of key values.

Hash Rings

The algorithm also needs to replicate items with these ranges from the old nodes to the new nodes.

The concept of a hash ring can also be extended to include the requirement that an item must be stored on multiple nodes.

When a new item is created, the **hash ring rules might indicate both a primary and a secondary copy of where an item is stored.**

If the node that contains the primary fails, the system can look up the node where the secondary item is stored.

Using replication to scale reads

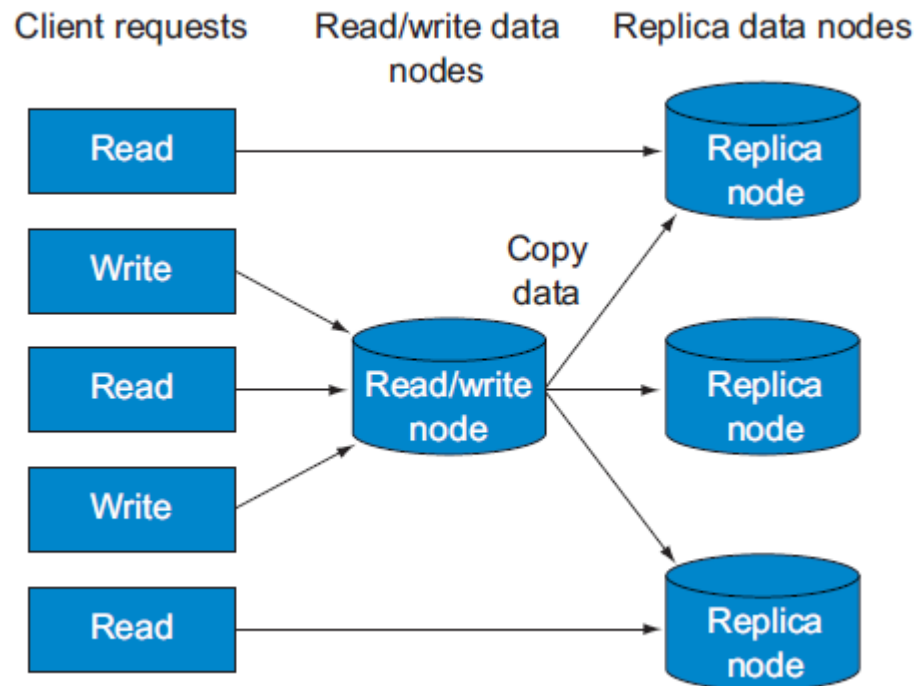
Replication strategy works well in most cases. There are only a few times when you must be concerned about the lag time between a write to the read/write node and a client reading that same record from a replica.

One of the most common operations after a write is a read of that same record.

If a client does a write and then an immediate read from that same node, there's no problem.

The problem occurs if a read occurs from a replica node before the update happens. This is an example of an inconsistent read.

Using replication to scale reads



All incoming client requests enter from the left. All reads can be directed to any node, either a primary read/write node or a replica node.

All write transactions can be sent to a central read/write node that will update the data and then automatically send the updates to replica nodes.

The time between the write to the primary and the time the update arrives on the replica nodes determines how long it takes for reads to return consistent results.

Using replication to scale reads

The best way to avoid this type of problem is to only allow reads to the same write node after a write has been done.

This logic can be added to a session or state management system at the application layer.

Almost all distributed databases relax database consistency rules when a large number of nodes permit writes.

If your application needs fast read/write consistency, you must deal with it at the application layer.

Letting the database distribute queries evenly to data nodes

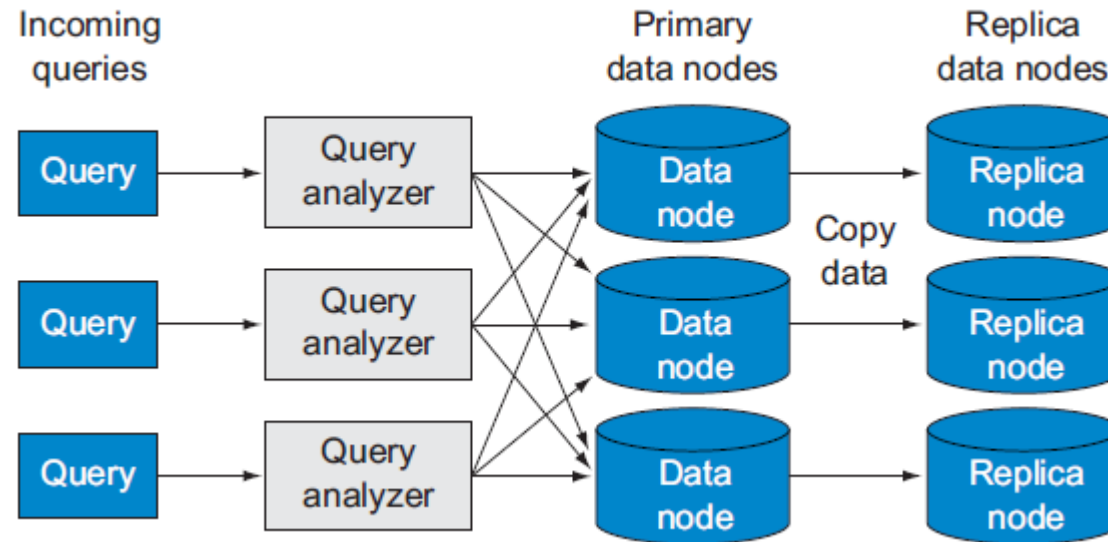
In order to get high performance from queries that span multiple nodes, it's important to separate the concerns of query evaluation from query execution.

The approach shown in next figure is moving the query to the data rather than moving the data to the query. This is an important part of NoSQL big data strategies.

NoSQL systems move the query to a data node, but don't move data to a query node.

In this instance, moving the query is handled by the database server, and distribution of the query and waiting for all nodes to respond is the sole responsibility of the database, not the application layer.

Distribute queries evenly



In this example, all incoming queries arrive at query analyzer nodes. These nodes then forward the queries to each data node.

If they have matches, the documents are returned to the query node.

The query won't return until all data nodes (or a response from a replica) have responded to the original query request.

If the data node is down, a query can be redirected to a replica of the data node.

Distribute queries evenly

This approach is somewhat similar to the concept of **federated search**.

Federated search takes a single query and distributes it to distinct servers and then combines the results together to give the user the impression they're searching a single system.

In some cases, these servers may be in different geographic regions.

In this case, **you're sending your query to a single cluster that's not only performing search queries on a single local cluster but also performing update and delete operations.**

Summary

This session will give the knowledge about

- NoSQL Systems to handle Big data